

Application Python

1) Créez datasetManager, le dossier racine de l'application

mkdir datasetManager

2) Créez le fichier main.py, point d'entrée de l'application

cd datasetManager

touch main.py

Partie 1 : Types de base, variables, Entrées et sorties

3) Demandez à l'utilisateur de saisir les métadonnées d'un dataset :

```
# 3- Demandez à l'utilisateur de saisir les métadonnées d'un dataset

nom = input("Nom du dataset : ")
domaine = input("Domaine : ")
lignes = int(input("Nombre de lignes : "))
colonnes = int(input("Nombre de colonnes : "))
taille = float(input("Taille en Mo : "))
format_dataset = input("Format (csv ou json) : ")
public = input("Dataset public (true ou false) : ")
```

Ce programme récupère les différentes informations nécessaires pour décrire un dataset :

- Nom : nom du jeu de données ;
- Domaine : secteur d'utilisation du dataset ;
- Lignes et colonnes : caractéristiques quantitatives ;
- Taille : poids du fichier en Mo ;
- format_dataset : type de fichier utilisé ;
- Public : indique si le dataset est accessible publiquement.

4) Affichez ensuite un résumé formaté

```
# 4-Affichez ensuite un résumé formaté.
print("\n==== Résumé du dataset =====")
print("Nom :", nom)
print("Domaine :", domaine)
print("Nombre de lignes :", lignes)
print("Nombre de colonnes :", colonnes)
print("Taille :", taille, "Mo")
print("Format :", format_dataset)
print("Public :", public)]
```

Cette partie affiche un résumé clair des informations saisies par l'utilisateur. Elle permet de vérifier que les données du dataset ont bien été enregistrées.

Partie 2 – Structures de contrôle

5) Créez un menu interactif (provisoire)

```
# Créez un menu interactif (provisoire)

choix = ""

while choix != "4":

    print("\n=====")
    print("1. Ajouter un dataset")
    print("2. Afficher les datasets")
    print("3. Rechercher")
    print("4. Quitter")
    print("=====")

    choix = input("Votre choix : ")

    if choix == "1":
        print("Ajout d'un dataset")

    elif choix == "2":
        print("Affichage des datasets")

    elif choix == "3":
        print("Recherche d'un dataset")

    elif choix == "4":
        print("Fermeture du programme")

    else:
        print("Choix invalide")
```

Cette partie introduit une **structure de contrôle** avec une boucle while et des conditions if / elif / else.

- La variable choix permet de récupérer la décision de l'utilisateur.
- La boucle while maintient le programme actif tant que l'utilisateur ne choisit pas l'option 4 (**Quitter**).
- Les conditions permettent d'exécuter une action différente selon le choix effectué.

Partie 3 – Dictionnaires

6) Créez un dictionnaire pour stocker les métadonnées de chaque dataset

```
# 6 Créez un dictionnaire pour stocker les métadonnées de chaque dataset
```

```
dataset = {  
    "nom": nom,  
    "domaine": domaine,  
    "lignes": lignes,  
    "colonnes": colonnes,  
    "taille": taille,  
    "format": format_dataset,  
    "public": public  
}  
  
print("\n==== Résumé du dataset =====")  
  
print(dataset)
```

Un dictionnaire permet de stocker les informations sous forme de **clé : valeur**.

Exemple :

- "nom" représente la clé qui identifie l'information.
- "Titanic" représente la valeur associée.

L'utilisation d'un dictionnaire rend les données plus faciles à manipuler, car toutes les informations du dataset sont regroupées dans une seule variable.

Partie 4 – Tuples

7) Créez un tuple contenant les domaines autorisés.

```
# 7- Créez un tuple contenant les domaines autorisés.  
  
domaines_autorises = (  
    "Santé",  
    "Finance",  
    "Agriculture",  
    "Transport",  
    "Education"  
)
```

Un tuple est une structure qui permet de stocker plusieurs valeurs qui ne doivent pas être modifiées.

Ici, il contient la liste des domaines acceptés par l'application.

8) Vérifiez que le domaine saisi, à la question 3, appartient au tuple

```
if domaine in domaines_autorises:  
    print("Domaine valide")  
else:  
    print("Domaine non autorisé")
```

L'opérateur in permet de vérifier si une valeur existe dans une liste ou un tuple.

Dans notre cas :

- si le domaine saisi existe dans domaines_autorises, il est accepté ;
- sinon, un message indique qu'il n'est pas autorisé.

Partie 5 – Listes

9) Créez une liste contenant les datasets. Chaque ajout est enregistré dans la liste

```
# 9- Créez une liste contenant les datasets. Chaque ajout est enregistré dans la liste

datasets = []

datasets.append(dataset)

print("\nListe des datasets :")
print(datasets)
```

Une liste permet de stocker plusieurs éléments dans une seule variable.

- crée une liste vide.
- ajoute le dictionnaire du dataset dans cette liste.

10) Ajoutez les fonctionnalités :

- **Ajouter**

```
def ajouter_dataset():

    print("\n=== Nouveau Dataset ===")

    nom = input("Nom : ")

    while True:
        domaine = input("Domaine : ").title()

        if domaine in DOMAINES:
            break
        else:
            print("Domaine invalide.")
            print("Domaines autorisés :", DOMAINES)

    lignes = int(input("Nombre de lignes : "))
    colonnes = int(input("Nombre de colonnes : "))
    taille = float(input("Taille (Mo) : "))

    while True:
        format_ds = input("Format (CSV/JSON) : ").upper()

        if format_ds in ("CSV", "JSON"):
            break
        print("Format invalide.")

    public = input("Public (true/false) : ").lower()

    if public == "true":
        public = True
    else:
        public = False
```

```
dataset = {
    "nom": nom,
    "domaine": domaine,
    "lignes": lignes,
    "colonnes": colonnes,
    "taille": taille,
    "format": format_ds,
    "public": public
}

datasets.append(dataset)

print("\nDataset enregistré avec succès !")
```

Cette fonction permet de saisir les informations d'un nouveau dataset (nom, domaine, nombre de lignes, colonnes, taille, format et visibilité). Elle vérifie que le domaine et le format sont valides, crée un dictionnaire contenant ces informations, puis l'ajoute à la liste datasets.

- **Trier**

```
def trier():

    datasets.sort(key=lambda d: d["nom"].lower())

    print("Datasets triés par nom.")
```

Cette fonction trie la liste des datasets par ordre alphabétique selon leur nom grâce à la méthode sort(). Une fois le tri terminé, un message confirme que les datasets ont été classés.

- **Rechercher**

```
def rechercher():
    recherche = input("Entrez le nom du dataset à rechercher : ")

    trouve = False
    for data in datasets:
        if data["nom"].lower() == recherche.lower():
            print("Dataset trouvé :", data)
            trouve = True

    if not trouve:
        print("Dataset introuvable.")
```

Cette fonction demande à l'utilisateur le nom d'un dataset, puis parcourt la liste pour le retrouver. Si le dataset existe, ses informations sont affichées ; sinon, un message indique qu'il est introuvable.

- **Modifier**

```
def modifier():
    nom = input("Nom du dataset à modifier : ").lower()

    for ds in datasets:
        if ds["nom"].lower() == nom:
            print("Nouvelles informations")

            ds["nom"] = input("Nom : ")
            ds["lignes"] = int(input("Nombre de lignes : "))
            ds["colonnes"] = int(input("Nombre de colonnes : "))
            ds["taille"] = float(input("Taille : "))

            print("Modification effectuée.")
            return

    print("Dataset introuvable.")
```

Cette fonction permet de modifier les informations d'un dataset existant. Après avoir recherché le dataset par son nom, elle demande les nouvelles valeurs et met à jour les informations correspondantes.

- **Supprimer**

```
def supprimer():
    nom = input("Nom du dataset à supprimer : ").lower()

    for ds in datasets:
        if ds["nom"].lower() == nom:
            datasets.remove(ds)
            print("Dataset supprimé.")
            return

    print("Dataset introuvable.")
```

Cette fonction recherche un dataset à partir de son nom. Si le dataset est trouvé, il est supprimé de la liste datasets. Sinon, un message indique que le dataset n'existe pas.

Partie 6 – Compréhensions (Listes et Dictionnaires)

11) Affichez les statistiques

```
# 11-Afficher les Statistiques
def statistiques():

    print("\n===== STATISTIQUES =====")

    if len(datasets) == 0:
        print("Aucun dataset.")
        return

    nb = len(datasets)

    total_lignes = sum(ds["lignes"] for ds in datasets)

    moyenne_colonnes = sum(ds["colonnes"] for ds in datasets) / nb

    publics = sum(1 for ds in datasets if ds["public"])

    prives = nb - publics

    csv = sum(1 for ds in datasets if ds["format"] == "CSV")

    json = sum(1 for ds in datasets if ds["format"] == "JSON")

    repartition = {
        domaine: sum(1 for ds in datasets if ds["domaine"] == domaine)
        for domaine in domaines_autorises
    }

    print("Nombre de datasets :", nb)
    print("Nombre total de lignes :", total_lignes)
    print("Nombre moyen de colonnes :", round(moyenne_colonnes, 2))
    print("Datasets publics :", publics)
    print("Datasets privés :", prives)
    print("Nombre de datasets CSV :", csv)
    print("Nombre de datasets JSON :", json)

    print("\nRépartition par domaine")

    for domaine, nombre in repartition.items():
        print(f"{domaine} : {nombre}")
```

Cette fonction permet de calculer et d'afficher les statistiques des datasets enregistrés dans la liste datasets. Elle vérifie d'abord si la liste est vide. Si des datasets existent, elle calcule le nombre total de datasets, le nombre total de lignes, le nombre moyen de colonnes, le nombre de datasets publics et privés, le nombre de fichiers au format CSV et JSON, ainsi que la répartition des datasets par domaine. Enfin, elle affiche toutes ces informations à l'écran.

Partie 7 – Fichiers

12) Créez le fichier datasets.csv

On n'a pas besoin de créer datasets.csv nous-même. C'est le programme ci-dessous qui va le créer automatiquement

- Sauvegarder les données

```
# 12- Créez le fichier datasets.csv
import csv
import os

def sauvegarder():

    with open("datasets.csv", "w", newline="", encoding="utf-8") as fichier:

        champs = [
            "nom",
            "domaine",
            "lignes",
            "colonnes",
            "taille",
            "format",
            "public"
        ]

        writer = csv.DictWriter(fichier, fieldnames=champs)

        writer.writeheader()

        writer.writerow(datasets)

    print("Sauvegarde effectuée.")
```

Cette fonction permet d'enregistrer les données des datasets dans un fichier **datasets.csv** afin de pouvoir les conserver après la fermeture du programme. Elle reçoit en paramètre une liste contenant les informations des datasets, ouvre le fichier en mode écriture, écrit les noms des colonnes puis ajoute chaque dataset ligne par ligne dans le fichier. À la fin, elle ferme le fichier et affiche un message indiquant que la sauvegarde est terminée.

- Recharger et afficher les données


```

def recharger():
    global datasets

    datasets = []

    with open("datasets.csv", "r", encoding="utf-8") as fichier:
        lecteur = list(csv.DictReader(fichier))

        if len(lecteur) == 0:
            print("Le fichier est vide.")

    with open("datasets.csv", "r", encoding="utf-8") as fichier:
        lecteur = csv.DictReader(fichier)

        for ligne in lecteur:
            ligne["lignes"] = int(ligne["lignes"])
            ligne["colonnes"] = int(ligne["colonnes"])
            ligne["taille"] = float(ligne["taille"])
            ligne["public"] = ligne["public"] == "True"

            datasets.append(ligne)

    print("Chargement terminé.")

```

Cette fonction permet de récupérer les données déjà enregistrées dans le fichier **datasets.csv**. Elle ouvre le fichier en mode lecture, lit chaque ligne du fichier et les ajoute dans une nouvelle liste appelée **datasets**. La première ligne contenant les titres des colonnes est ignorée. Une fois toutes les données récupérées, le fichier est fermé et la fonction retourne la liste des datasets chargés pour pouvoir les utiliser dans le programme.

Partie 8 – Exceptions

13) Gérez ces exceptions pour que le programme continue de fonctionner.

Les erreurs qu'on veut résoudre :

- L'utilisateur saisit un texte au lieu d'un nombre

```

try:
    choix = int(input("Entrez votre choix : "))
except ValueError:
    print("Erreur : veuillez saisir un nombre.")

```

Le programme essaie de convertir la saisie en nombre. Si l'utilisateur entre un texte, l'exception **ValueError** est détectée et un message d'erreur est affiché au lieu d'arrêter le programme.

- **Le fichier n'existe pas**

```
def recharge():  
    datasets = []  
  
    try:  
        fichier = open("datasets.csv", "r", encoding="utf-8")  
        lecteur = csv.reader(fichier)  
  
        next(lecteur)  
  
        for ligne in lecteur:  
            datasets.append(ligne)  
  
        fichier.close()  
  
        print("Données rechargées avec succès.")  
  
    except FileNotFoundError:  
        print("Erreur : le fichier datasets.csv n'existe pas.")  
  
    return datasets
```

Lors de la recharge des données, si le fichier datasets.csv n'existe pas, Python provoque une erreur **FileNotFoundError**.

Si le fichier n'a jamais été créé ou a été supprimé, le programme affiche un message d'information et continue son exécution.

- **Le dataset recherché n'existe pas**

```
def rechercher(datasets, nom):  
    trouve = False  
  
    for dataset in datasets:  
        if dataset[0] == nom:  
            print(dataset)  
            trouve = True  
  
    if trouve == False:  
        print("Erreur : dataset introuvable.")
```

Lors d'une recherche, si le nom du dataset n'est pas trouvé dans la liste, on utilise une condition pour éviter une erreur.

La variable trouve permet de vérifier si un résultat existe. Si aucun dataset ne correspond au nom recherché, le programme affiche un message au lieu de continuer avec une valeur inexistante.

- Le fichier est vide

```
def recharge():
    datasets = []

    try:
        fichier = open("datasets.csv", "r", encoding="utf-8")
        lecteur = csv.reader(fichier)

        entete = next(lecteur)

        for ligne in lecteur:
            datasets.append(ligne)

        if len(datasets) == 0:
            print("Le fichier est vide.")

        fichier.close()

    except FileNotFoundError:
        print("Le fichier n'existe pas.")

    return datasets
```

Si le fichier existe mais ne contient aucune donnée, la lecture peut provoquer un problème avec next().

Après la lecture du fichier, on vérifie si la liste datasets contient des éléments. Si elle est vide, le programme informe l'utilisateur.

Partie 9 – Fonctions

14) Refactorisez le programme en créant les fonctions suivantes :

```
afficher_menu()
pass
ajouter_dataset()
pass
afficher_datasets()
pass
supprimer_datasets()
pass
rechercher_datasets()
pass
trier_datasets()
pass
modifier_datasets()
pass
sauvegarder()
pass
recharger()
pass
statistiques()
```

La Partie 9 transforme le programme en une application mieux organisée.

- Avant :

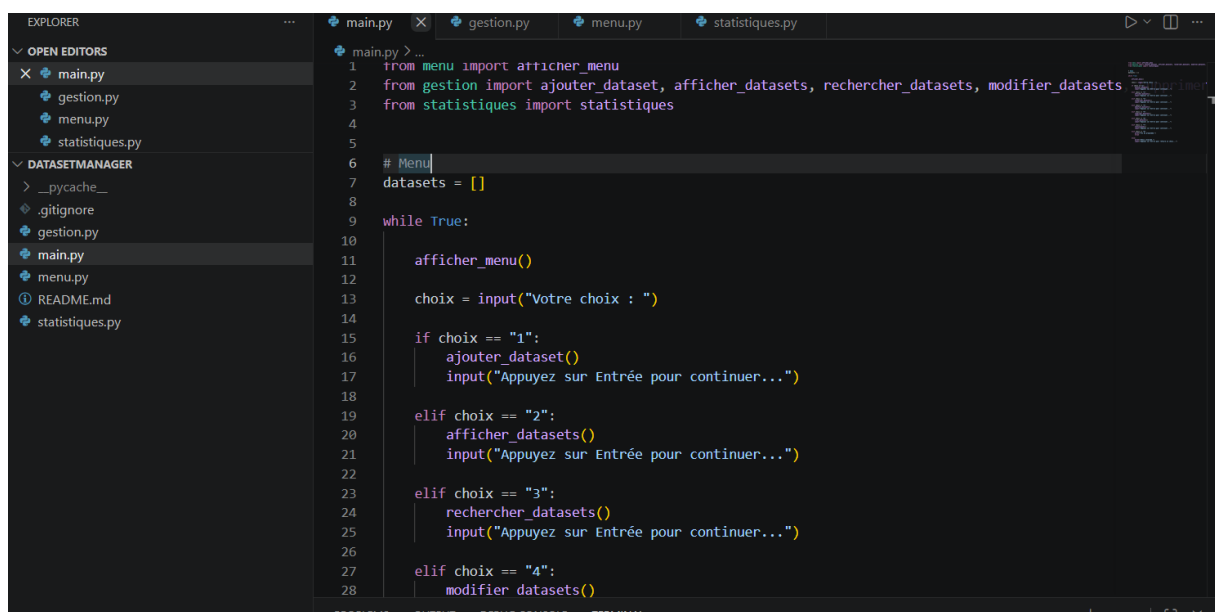
Tout le code était mélangé dans main.py.

- Après :

Chaque action possède sa propre fonction, le programme principal devient plus simple à lire et les fonctions pourront facilement être déplacées dans des modules à la prochaine étape.

Partie 10 – Modules

15) Pour éviter des problèmes de maintenance, répartissons le code dans plusieurs modules.



```

EXPLORER
  OPEN EDITORS
    main.py
  DATASETMANAGER
    __pycache__
    .gitignore
    gestion.py
    main.py
    menu.py
    README.md
    statistiques.py

main.py
1 from menu import afficher_menu
2 from gestion import ajouter_dataset, afficher_datasets, rechercher_datasets, modifier_datasets
3 from statistiques import statistiques
4
5
6 # Menu
7 datasets = []
8
9 while True:
10
11     afficher_menu()
12
13     choix = input("Votre choix : ")
14
15     if choix == "1":
16         ajouter_dataset()
17         input("Appuyez sur Entrée pour continuer...")
18
19     elif choix == "2":
20         afficher_datasets()
21         input("Appuyez sur Entrée pour continuer...")
22
23     elif choix == "3":
24         rechercher_datasets()
25         input("Appuyez sur Entrée pour continuer...")
26
27     elif choix == "4":
28         modifier_datasets()
  
```

La séparation en modules permet d'avoir un code plus propre de retrouver facilement une fonctionnalité de modifier une partie sans toucher au reste du programme.

Maintenant :

- **main.py** → lance l'application ;
- **menu.py** → gère l'affichage du menu ;
- **gestion.py** → gère les datasets ;
- **statistiques.py** → calcule les statistiques

Partie 11 – Packages

16) Découpez l'application en packages

En Python, un package est simplement un dossier qui contient des modules Python et, dans le cadre de notre exercice, un fichier `__init__.py`.

- On utilise la commande **mkdir** pour créer les dossiers

```
moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager (main)
• $ mkdir datasets

moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager (main)
• $ mkdir interface

moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager (main)
• $ mkdir stockage

moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager (main)
• $ mkdir data
```

- On utilise la commande **touch** pour créer les fichiers (txt,json,csv,py)

```
moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager/data (main)
• $ touch datasets.csv

moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager/data (main)
• $ touch datasets.json
```

- On utilise la commande **mv** pour déplacer les fichiers vers les dossiers

```
moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager (main)
• $ mv gestion.py datasets

moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager (main)
• $ mv statistiques.py datasets

moham@MOHAMED-N-DIAO MINGW64 ~/Documents/datasetManager (main)
• $ mv menu.py interface
```

Partie 12 : Bonus

- **Exporter les statistiques dans un fichier**

Créer un fichier (statistiques.txt) contenant les statistiques.

- **Recherche par plusieurs critères**

Au lieu de rechercher uniquement par le nom, permettre de rechercher par :

- ✓ Domaine
- ✓ Format
- ✓ Public/privé
- ✓ Nombre minimum de lignes

- **Affichage sous forme de tableau**

Nom : Titanic

Domaine : Transport

Format : CSV

Afficher :

Nom	Domaine	Format	Lignes	Colonnes
Titanic	Transport	CSV	891	12
Covid19	Santé	JSON	55000	18
Banque	Finance	CSV	120000	9